Using Ansible to Update AWS Instances

Caleb Quisenberry

University of Advancing Technology

July 21, 2024

Author Note

This paper was prepared for NTS336 Cloud Security, taught by Professor Aaron Jones.

Abstract

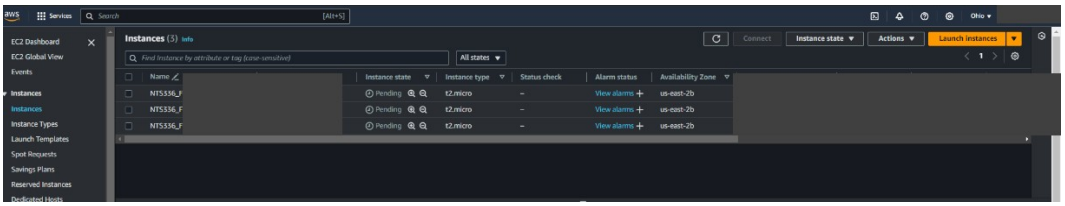
This report details the process of using Ansible to update and upgrade multiple AWS EC2 instances. It describes the steps taken to create and set up the EC2 instances, the purpose and structure of the Ansible playbook, the benefits of using Ansible for server management, and the challenges encountered during the assignment.

Using Ansible to Update AWS Instances

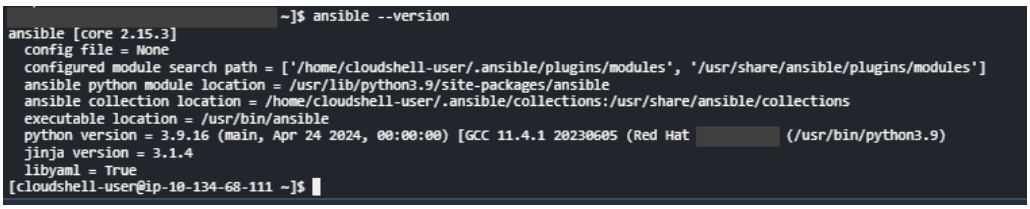
Automation tools like Ansible simplify the management of multiple servers by allowing for efficient and consistent configuration management. This assignment involved launching three EC2 instances, setting them up for Ansible management, and using an Ansible playbook to update and upgrade the instances.

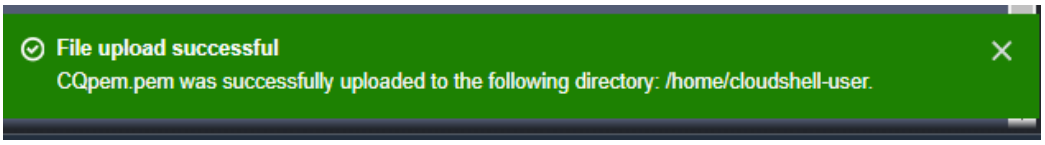
Create and Launch EC2 Instances

To create and set up the EC2 instances for Ansible management, I began by logging into the AWS Management Console. Navigating to the EC2 Dashboard, I launched three instances using the Amazon Linux 2 AMI, selecting the t2.micro instance type to remain within the AWS free tier. I configured the instances with appropriate storage and security group settings, ensuring that SSH access (port 22) was allowed, and assigned a key pair for secure SSH access.

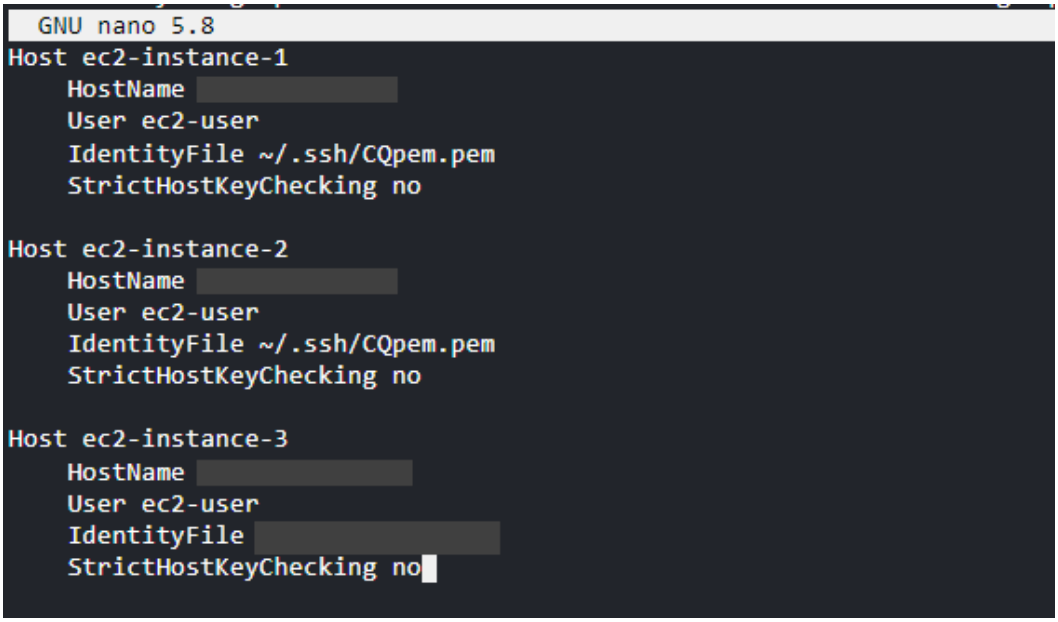


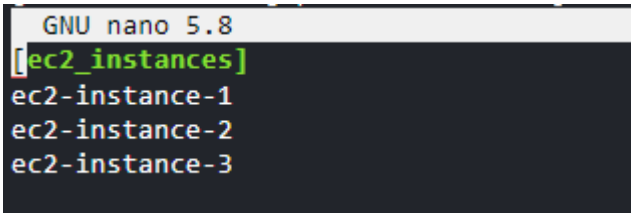
After launching the instances, I used AWS CloudShell to upload the SSH key and connect to each instance to verify connectivity and perform the initial setup. I then updated the package list and installed Ansible on CloudShell. This step gave me a bit of trouble as I was skipping the upload part and attempting to route to local storage.





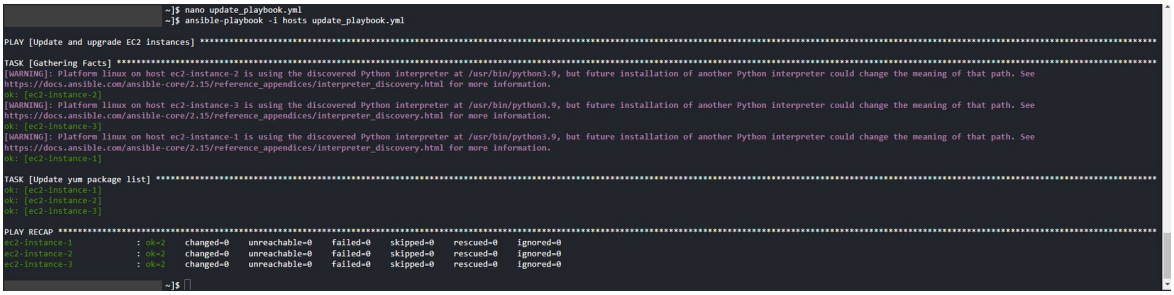
With Ansible installed, I proceeded to configure the inventory and playbook. I created an inventory file listing the private IP addresses of the EC2 instances. Then, I developed an Ansible playbook designed to update and upgrade the EC2 instances using the yum package manager. This setup ensured that the instances were properly configured and ready for automated management using Ansible. My initial attempts failed, as I had not uploaded the pem file. I also initially was using apt as a package manager and the choice of Amazon Linux required the use of yum. The following screenshots show the modification to the ssh config file, and the inventory and playbook files.





```
  GNU nano 5.8
---
- name: Update and upgrade EC2 instances
  hosts: ec2_instances
  become: yes
  tasks:
    - name: Update yum package list
      yum:
        name: '*'
        state: latest
```

The following screenshot illustrates the successful operation of the Ansible playbook.



Purpose and Structure of the Ansible Playbook

The purpose of the Ansible playbook was to automate the process of updating and upgrading the software packages on the EC2 instances. The playbook was structured with an inventory file that listed the target instances and a playbook file that defined the tasks to update and upgrade the instances. The inventory file included the private IP addresses of the EC2 instances, while the playbook file contained tasks to update the package list and upgrade all packages using the package manager.

Benefits of Using Ansible for Managing Multiple Servers

Ansible offers several benefits for managing multiple servers. Its simplicity and ease of use stem from its straightforward YAML syntax, which makes writing and understanding playbooks easy. The agentless architecture of Ansible eliminates the need for installing agents on

the managed nodes, thereby reducing overhead. Ansible's idempotency ensures that running the playbooks multiple times does not alter the system if it is already in the desired state. Additionally, Ansible is highly scalable, making it easy to manage a large number of servers with minimal configuration changes.

Challenges and Resolutions

During the assignment, several challenges were encountered and resolved. One major challenge was SSH key recognition. Initial attempts to run the playbook failed due to an incorrect SSH key path and host key verification issues. This was resolved by uploading the SSH key to CloudShell, moving it to the correct directory, adjusting the permissions, and creating an SSH configuration file to handle host key verification. Another challenge was the compatibility of the package manager. My original playbook used apt, which is not available on Amazon Linux 2. This was resolved by modifying the playbook to use yum, the appropriate package manager for Amazon Linux 2.

Conclusion

This assignment provided firsthand experience with using Ansible to manage multiple AWS EC2 instances. By automating the update and upgrade process, Ansible demonstrated its effectiveness in simplifying server management tasks. The challenges encountered were addressed through proper configuration and adjustments, highlighting the importance of understanding the underlying infrastructure and tools. This has been a valuable course that has provided real-world experiences in distributed computing theories and practices.

References

Amazon Web Services. (2019). *What Is Amazon EC2? - Amazon Elastic Compute Cloud*. Amazon.com. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html

Bennett, T. (n.d.). *What is Idempotency?* Blog.dreamfactory.com. Retrieved July 21, 2024, from https://blog.dreamfactory.com/what-is-idempotency

Getting started with Ansible — Ansible Documentation. (n.d.). Docs.ansible.com. https://docs.ansible.com/ansible/latest/getting_started/index.html